

Log In ▾

The wiki page is under active construction, expect bugs.

Operační systémy a jejich architektury. Systémová volání, vlákna, procesy. Správa virtuální a fyzické paměti, souborové systémy. Bezpečnost, virtualizace.

vector2 ▾

B4B35OSY Webové stránky předmětu

- **Systémová volání** – jak je implementována ochrana paměti jádra, jak se předávají parametry a data ze systémových volání, rozdíl mezi mikro jádrem a monolitickým jádrem.
- **Vlákna, procesy** – jak se vytvoří proces, jak lze předat data mezi procesy. Jaký je rozdíl mezi vlákny a procesy, která data sdílejí různá vlákna jednoho procesu (registry, zásobník, lokální proměnné, globální proměnné, dynamicky alokované proměnné).
- **Synchronizace vláken** – jaké jsou problémy při paralelním přístupu ke sdíleným datům, jaké existují synchronizační prostředky, co je to deadlock, kdy může nastat a jak se lze deadlocku vyhnout.
- **Správa virtuální a fyzické paměti** – co je a jak vypadá stránkovací tabulka, jaké jsou zásadní nevýhody stránkování, TLB (translation-lookaside-buffer), víceúrovňové stránkování v 32-bitovém a 64-bitovém systému, odkládání stránek na disk, algoritmy výběru oběti, metoda copy-on-write.
- **Souborové systémy** – jaké typy souborových systémů znáte, který je vhodný pro sekvenční čtení a který pro náhodné čtení souborů. Vysvětlete základní souborové systémy: FAT, systémy založené na inodech a systémy založené na extendech. Žurnálování – základní princip, kdy mohou vzniknout v souborovém systému chyby, jaké jsou úrovně žurnálování a jeho nevýhody.
- **Bezpečnost** – co je Trusted Computing Base, základní metody řízení přístupu, jak se provádí útok na přetečení zásobníku, jak se lze takovému útoku bránit.
- **Virtualizace** – softwarová virtualizace, metoda trap-and-emulate, virtualizace systémového volání, virtualizace stránkovacích tabulek, hardwarově asistovaná virtualizace.

1. Systémová volání

jak je implementována ochrana paměti jádra, jak se předávají parametry a data ze systémových volání, rozdíl mezi mikro jádrem a monolitickým jádrem.

Systémová volání jsou rozhraní mezi uživatelským a jádrovým režimem. Umožňují uživatelským aplikacím komunikovat s jádrem operačního systému a využívat jeho služby. Systémová volání se obvykle implementují pomocí přerušení nebo speciálních instrukcí (SYSCALL či SYSENTER), které přepínají procesor do jádrového režimu.

Jak se volají systémová volání

Jádro implementuje tzv. **ABI (Application Binary Interface)**, což je rozhraní, které definuje, jakým způsobem se volají systémová volání a jak se předávají parametry. Systémová volání mají čísla, která se uloží do registru (např. **EAX** v x86 architektuře) a parametry se předávají pomocí dalších registrů nebo na zásobník. Po provedení systémového volání jádro vrátí výsledek do registru a přepne procesor zpět do uživatelského režimu. Pokud potřebujeme předat větší množství dat, jako jsou struktury nebo pole, používá se ukazatel na paměť, který se předává jako parametr. Jádro pak může přistupovat k těmto datům přímo.

Předávání parametrů a návratových hodnot

Parametry systémového volání se předávají různými způsoby podle architektury a konkrétního API:

- **Jednoduché hodnoty** (čísla, příznaky, velikosti atd.) se ukládají do registrů – např. v Linuxu na x86-64 se používají registry **rđi, rsi, rdx, r10, r8, r9**.
- **Složitější struktury nebo pole** se předávají pomocí **ukazatelů**, které jádro validuje a dereferencuje.
- **Výsledky systémových volání** se obvykle vrací do jednoho registru (např. **rax**) – typicky návratový kód nebo počet zpracovaných bajtů. V případě chyby se vrací záporný návratový kód (např. `-EINVAL`, `-EPERM`, ...).

Předávání přes paměť je bezpečnější, protože jádro ověří přístupová práva (např. zda proces může číst/zapisovat do dané oblasti).

Ochrana paměti jádra

Hlavním způsobem ochrany paměti jádra je použití **virtuální paměti**. Každý proces má svůj vlastní virtuální adresní prostor, což znamená, že každý proces vidí svou vlastní paměť a nemůže přistupovat k paměti jiných procesů nebo jádra. To se provádí pomocí **stránkování**, které mapuje virtuální adresy na fyzické adresy.

V moderních architekturách procesory implementují tzv. **privilegované režimy** (*rings of privilege*). Jádro OS běží v nejvyšším privilegovaném režimu (**ring 0** v x86), s plným přístupem k hardwaru a paměti. Uživatelské aplikace běží v méně privilegovaném režimu (**ring 3** v x86) s omezeným přístupem. Přechod z uživatelského do jádrového režimu je řízen např. přes systémová volání.

- **Důsledek:** Uživatelský kód nemůže přímo přistupovat k jádrové paměti – případný pokus vede k výjimce (segfault).

Pozn.: Intel v r. 2024 oznámil útlum podpory pro ring 1 a 2 v x86, jelikož je moderní OS nevyužívají.

Mikro jádro vs. monolitické jádro

Mikro jádro a monolitické jádro jsou dva základní přístupy k návrhu operačního systému:

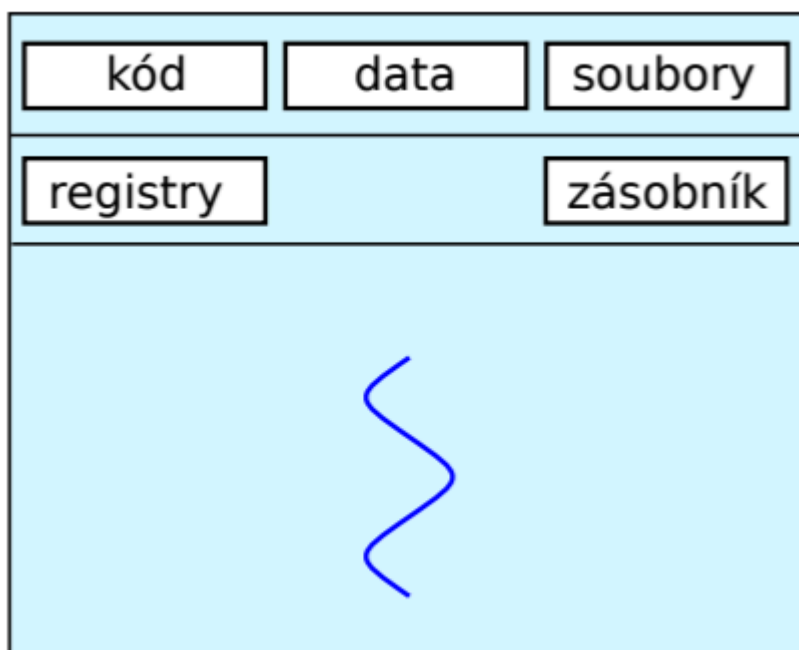
- **Mikro jádro:**
 - **Princip:** Minimalizuje kód běžící v jádrovém režimu. Většina OS služeb (např. ovladače zařízení, souborové systémy) běží jako oddělené procesy v uživatelském prostoru.
 - **Výhody:**
 - Vyšší modularita a flexibilita.
 - Lepší bezpečnost a stabilita (izolace chyb – pád jedné služby neohrozí celé jádro).
 - Snazší vývoj, testování a přenositelnost jednotlivých komponent.
 - Menší velikost samotného jádra.
 - **Nevýhody:**
 - Potenciálně nižší výkon kvůli režii meziprocesové komunikace (IPC) mezi službami a jádrem.
 - Složitější návrh mechanismů komunikace mezi procesy.
- **Monolitické jádro:**
 - **Princip:** Většina OS služeb (včetně ovladačů, správy paměti, plánovače procesů) běží v jednom velkém programu v jádrovém režimu.
 - **Výhody:**
 - Vyšší výkon, protože komunikace mezi komponentami probíhá pomocí přímých volání funkcí bez režie IPC.
 - Jednodušší návrh (alespoň zpočátku, pro základní funkce).
 - **Nevýhody:**
 - Nižší modularita.
 - Větší komplexita kódu s rostoucí velikostí, což ztěžuje údržbu a ladění.
 - Chyba v jedné části může vést k pádu celého systému.
 - Obtížnější přenositelnost.
 - Větší bezpečnostní riziko, pokud je kompromitována část jádra.
- **Hybridní jádro:**
 - Kombinuje prvky mikro jádra a monolitického jádra.
 - Některé části jádra jsou implementovány jako samostatné procesy (jako v mikro jádře), zatímco jiné části běží přímo v jádrovém režimu (jako v monolitickém jádře).

2. Vlákna a procesy

jak se vytvoří proces, jak lze předat data mezi procesy. Jaký je rozdíl mezi vlákny a procesy, která data sdílí různá vlákna jednoho procesu (registry, zásobník, lokální proměnné, globální proměnné, dynamicky alokované proměnné)

Vlákna

- **Vlákno** je lehká plánovací jednotka – sekvence instrukcí běžících na CPU.
- Vlákna jednoho procesu **sdílí** jeho adresní prostor, heap, globální a statické proměnné, otevřené soubory a další kernel-objekty.
- Každé vlákno má **vlastní**:
 - kontext CPU (registry včetně PC, SP, ...),
 - zásobník pro lokální proměnné a návratové adresy,
 - thread-local storage (TLS),
 - **TCB (Thread Control Block)** – kam se při přepnutí ukládá kontext vlákna.
- Tradiční „monolitický“ proces = proces s jediným vláknem.
- **Stavy vláken**: Running (běží), Ready (připravené), Blocked/Waiting (čeká), Terminated (ukončené).
- Přepnutí (context switch) uloží registry + stack pointer do TCB a načte kontext jiného vlákna.
- OS plánuje vlákna podobně jako procesy; implementace může být kernel-level, user-level nebo hybridní (např. NPTL v Linuxu).
- **PCB vs. TCB**
 - **PCB (Process Control Block)** – jádrová struktura **procesu**:
 - PID, stav procesu, ukazatel na tabulky stránek (adresní prostor), otevřené soubory, bezpečnostní údaje (UID, GID, capabilities), signálová maska, limity zdrojů, statistiky CPU času atd.
 - **TCB (Thread Control Block)** – lehčí záznam **vlákna**:
 - uložené registry (PC, SP ...), ukazatel na vlastní zásobník, TLS pointer, stav vlákna, priorita, CPU afinity, statistiky.
 - **Vztah**: PCB udržuje seznam všech TCB patřících procesu.
 - Přepnutí mezi vlákny téhož procesu mění jen TCB (rychlejší); přepnutí na vlákno jiného procesu ukládá navíc PCB (pomalejší).



jednovláknový proces



vícevláknový

Sdílená vs. privátní data ve vícevláknovém procesu

Kategorie	Sdílí všechna vlákna	Jedinečné pro vlákno
_____	_____	_____
Kód (text) & globální data	✓	✗
Heap (dynamicky alokovaná paměť)	✓	✗
Otevřené popisovače souborů	✓	✗
Programový čítač & registry	✗	✓
Zásobník (stack)	✗	✓
Thread-local storage (TLS)	✗	✓

Přednosti:

- Vlákno se vytvoří i ukončí rychleji než proces.
- Přepínání mezi vlákny je rychlejší než mezi procesy.
- Dosáhne se lepší strukturalizace programu.

Realizace:

- knihovna **PThread**
- v jazyce Java pomocí třídy **Thread**

Procesy

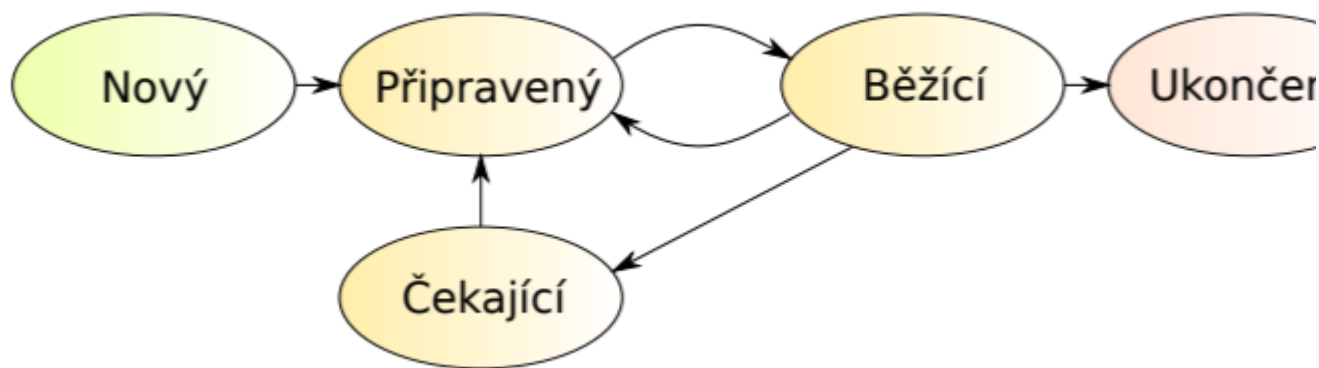
- **Program** – soubor (např. na disku) definovaného formátu obsahující instrukce, data a údaje potřebné k inicializaci procesu.
- **Proces** – spuštěný program, spravovaný operačním systémem:
 - má vlastní virtuální adresní prostor, otevřené soubory, systémové prostředky, ...,
 - může obsahovat více vláken (multithreadovaný proces),
 - je identifikovatelný **PID** (Process IDentifier),
 - paměť procesu: ``.text`` (kód), ``.data`` (globální), ``.stack``, heap.
- Proces je objekt s vlastním kontextem:
 - registry procesoru (čítač instrukcí, ukazatel zásobníku, příznaky FLAGS, uživatelské registry, FPU registry)
 - přidělený paměťový prostor
 - seznam otevřených souborů
 - komunikační kanály (pipes, sockets...)
 - použitá paměť: Zásobník – ``.stack``, Data – ``.data``, Program – ``.text``

Vytváření procesů a předávání dat

- Proces vytváří nový proces voláním systémového volání **fork()**.
 - vznikne téměř identická kopie rodičovského procesu.
 - rozdíly: návratová hodnota (0 pro dítě, PID pro rodiče), nový PID.
- Dítě může použít **exec()** k přepsání svého kódu jiným programem.
- Předání dat mezi procesy:
 - pomocí **sdílené paměti** (např. ``.shm_open``, ``.mmap``)
 - **potrubí (pipes)** – unidirekcionální datový tok
 - **sockets** – obousměrná komunikace, i mezi různými stroji
 - **signály** – pro jednoduchou asynchronní notifikaci

- **souborový systém** – zápis do souboru čitelný jiným procesem

Stavy procesů



- Přepnutí mezi procesy nastává po přerušení, výjimce nebo explicitním uvolnění CPU.
- Proces může čekat v různých frontách: na CPU, na I/O, na synchronizaci, na alokaci paměti atd.

Meziprocesní komunikace

Přehled meziprocesní komunikace

Název	Anglicky
Signál	Signal
Roura	Pipe
Pojmenovaná roura	Named pipe
Soubor mapovaný do paměti	Memory-mapped file
Sdílená paměť	Shared memory
Semafor	Semaphore
Zasílání zpráv	Message passing
Soket	Socket

- IPC je klíčová pro spolupráci procesů.
- Výběr prostředku závisí na požadavcích (rychlost, bezpečnost, rozsah).

3. Synchronizace vláken

jaké jsou problémy při paralelním přístupu ke sdíleným datům, jaké existují synchronizační prostředky, co je to deadlock, kdy může nastat a jak se lze deadlocku vyhnout.

- cílem je zabránit současnému přístupu více vláken do kritické sekce programu

Problémy při paralelním přístupu

- Deadlock – situace, kdy dva nebo více procesů čekají na uvolnění zdrojů, které jsou drženy jinými procesy, a žádný z nich nemůže

- **Data race** – situace, kdy výsledek operace závisí na pořadí provedení vláken, což může vést k nečekaným výsledkům.
- **False sharing** – situace, kdy více vláken přistupuje k různým částem stejného cache řádku, což může vést k neefektivnímu využití cache a snížení výkonu. (není kritické, ale může hodně zpomalit)

Problémy při paralelním přístupu

- **Deadlock** – situace, kdy dva nebo více procesů čeká na uvolnění zdrojů, které jsou drženy jinými, a žádný nemůže pokračovat.
- **Data race** – výsledek výpočtu závisí na pořadí přístupu k proměnným → nekonzistentní chování.
- **False sharing** – více vláken přistupuje k různým částem stejného cache-řádku → výkonnostní problém (neporušuje správnost, ale zpomaluje).

Deadlock

Deadlock (uváznutí) nastává, když skupina vláken čeká na zdroje způsobem, který vytvoří cyklus a nikdo nemůže pokračovat.

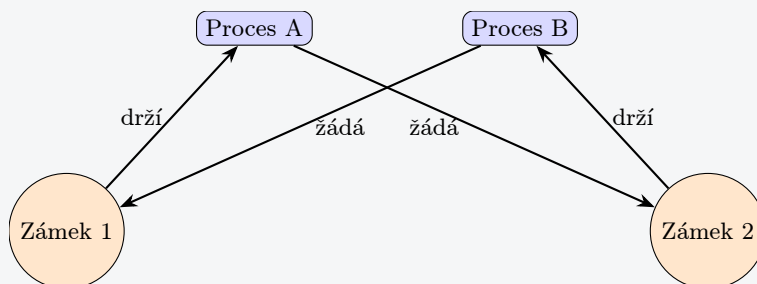
Coffmanovy podmínky pro vznik deadlocku:

1. **Vzájemné vyloučení** – zdroj může držet jen jedno vlákno.
2. **Hold and wait** – vlákno drží jeden zdroj a čeká na další.
3. **Neodnímatelnost** – zdroje nelze násilně odebrat.
4. **Cyklické čekání** – vznikne kruh, kde každé vlákno čeká na zdroj jiného.

Pokud všechny čtyři podmínky platí zároveň, vznikne deadlock.

Jak se deadlocku vyhnout:

- **Prevence** – např. nepovolíme Hold-and-Wait nebo nastavíme globální pořadí zámeků.
- **Vyhýbání (Avoidance)** – algoritmus bankéře: systém ověřuje, zda přidělením zdroje nevznikne nebezpečný stav.
- **Detekce + obnova** – detekce cyklů v grafu čekání; obnova ukončením nebo restartem procesu.



Synchronizační prostředky

- **Mutex (mutual exclusion)** – zámek, který umožňuje pouze jednomu vláknu přístup do kritické sekce. Pokud jedno vlákno zámek získá, ostatní musí čekat, dokud ho neuvolní.
- **Semafor** – celočíselná proměnná s operacemi:
 - **wait** – pokud je hodnota ≤ 0 , vlákno čeká; jinak se hodnota sníží.
 - **signal** – zvýší hodnotu a probudí čekající vlákno, pokud existuje.
 - Na rozdíl od mutexu může semafor zaručit, že se vlákno *vždy* časem dostane do kritické sekce; u mutexu může dojít ke *hladovění*.
- **Podmínkové proměnné (condition variables)** – slouží k čekání na konkrétní stav:
 - Vlákno držící mutex otestuje predikát (např. "fronta není prázdná").
 - Pokud predikát neplatí, vlákno se přesune do čekací fronty a *atomicky* uvolní mutex.
 - Po splnění predikátu může jiné vlákno zavolat `signal` (probudí jedno) nebo `broadcast` (všechna).
- **Hlavní operace (`pthread` API):**

- `pthread_cond_wait(&cond, &mutex)` – atomicky odemkne mutex, uspí vlákno, po probuzení mutex znovu zamkne.
- `pthread_cond_signal(&cond)` – probudí jedno čekající vlákno.
- `pthread_cond_broadcast(&cond)` – probudí všechna čekající vlákna.
- **Spurious wake-ups:** vlákno se může probudit i bez `signal` → **vždy testuj predikát v `while` smyčce**, ne v `if`.

```
pthread_mutex_lock(&m);
while (queue_empty)           // test v *while*!
    pthread_cond_wait(&cond, &m); // uvolní m, uspí, znovu zamkne m
dequeue_item();
pthread_mutex_unlock(&m);
```

• Rozdíl vůči semaforu:

- Semafor nese vlastní čítač; `wait` ho dekrementuje a když je > 0 , hned pokračuje → nestrácí signály.
- Cond-proměnná **nepamatuje historii**: pokud proces zavolá `signal`, když nikdo nečeká, signál se **ztratí**. Predikát musí být ve sdílené proměnné chráněné mutexem.

• Typické vzory použití:

- **Rendez-vous / producent–konzument** (fronta úloh)
- **Barrier** (vlákna čekají, než všechna dosáhnou určitého bodu)
- **Event flag** (čekání na vznik/neexistenci souboru, dokončení I/O atd.)

• Výhody: umožňuje spaní bez aktivního čekání, přesné buzení jen když je co dělat.

• Nevýhody: vyžaduje disciplínu (predikát v `while` + mutex), signály se mohou ztrácet, pokud nejsou korektně použity.

• Spin-lock – zámek, který místo uspání **aktivně točí procesor** (busy-wait), dokud se neuvolní. Hodí se jen pro opravdu krátké kritické sekce.

• Princip (krok za krokem)

- **Sdílená proměnná `lock`** je inicializována na `0` = neobsazeno.
- Vlákno provede ***atomickou*** instrukci `test-and-set` (TAS) nebo `compare-and-swap` (CAS)

• Výsledek atomické operace

- návratová `0` → zámek byl volný, vlákno ho právě získalo ⇒ vstup do kritické sekce;
- návratová `1` → zámek je držen někým jiným ⇒ vlákno **spinuje**: opakuje krok 2 (obvykle s `pause` či exponenciálním back-off, aby nezahltilo sběrnici).

• Uvolnění: vlákno po dokončení kritické sekce prostým zápisem nastaví `lock = 0`. (Operace zápisu je pro uvolnění bezpečná, protože z `1` → `0` není podmíněná.)

- U vícejádrových CPU se přidá ***release memory fence*** (`mfence`, `std::atomic_thread_fence(std::memory_order_release)`), aby se změny v kritické sekci ***nejprve*** propagovaly do paměti.

TAS: „ulož do `lock` hodnotu 1 a vrať mi předchozí obsah“.

```
tas:    mov  $1, %eax           ; pokusím se zapsat 1
        xchg %eax, lock        ; ATOMICKY výmění registr ↔ paměť
        ; pokud %eax == 0 → zámek byl volný a mám ho
```

CAS: „pokud je `lock == 0`, zapiš 1; jinak nedělej nic a dej mi současnou hodnotu“.

```
bool acquired = __sync_bool_compare_and_swap(&lock, 0, 1);
```

Tyto instrukce jsou **atomické**, protože CPU uzamkne cache-line / sběrnici a zaručí, že během operace ***nikdo jiný nemůže mezitím měnit* tutéž buňku paměti**.

• Proč „atomické nastavení“?

- Kdybychom dělali `if(lock==0) lock=1;`, dva thready mohou ***současně*** přečíst `0` a oba vstoupit – **race condition**.
- Atomická instrukce provede **čtení, test i zápis jako jedinou nepreemptovatelnou transakci**; hardware garantuje, že jen jeden zvítězí.

- **Kdy se vyplatí**

- Kritická sekce $\leq \sim 100$ CPU cyklů: inkrement globálního čítače, push/pop z velmi krátkého freelistu.
- OS kernel & interrupt context, kde uspání není možné.
- Vícejádrové systémy, kde je šance, že držitel zámku běží paralelně a brzy skončí.

- **Nevýhody / na co si dát pozor**

- Na jednojádro může spin-lock místo zrychlení vyvolat *hladovění* (drží CPU a nepustí vlákno, které má zámek uvolnit).
- Dlouhé držení zámku = plýtvání CPU → raději mutex, který vlákno uspí.
- Bez férového algoritmu (ticket, MCS) může některé vlákno točit výrazně déle (*starvation*).

- **Monitor** – „třída + vestavěný zámek + čekací fronta“, která **zapouzdřuje sdílená data a pravidla používání**:

- Voláš-li *veřejnou metodu* monitoru, **vstupuješ do monitoru** – systém ti **automaticky zamkne vnitřní mutex**.
- Žádné jiné vlákno se dovnitř nedostane, dokud z metody nevrátíš – tím se zámek zase uvolní.
 - Všechny změny dat uvnitř monitoru jsou atomické vůči ostatním vláknům.
- Potřebuje-li metoda čekat na nějaký **stav** (např. „fronta není prázdná“), zavolá `wait(cond)`
 - vlákno se přesune do **čekací fronty** dané podmínky a **současně opustí monitor** (uvolní vnitřní zámek),
 - jiná vlákna mohou stav změnit.
- Když někdo zavolá `signal(cond)`, jedno čekající vlákno se probudí, znovu vstoupí do monitoru a pokračuje – zámek už drží.
- Důsledek: **uživatel monitoru nikdy ručně nemanipuluje se zámkem ani s podmínkovými proměnnými** – ty jsou schované „pod kapotou“.
- Programátor píše jen logiku metod a test predikátů.
- **Rozdíl proti „mutex + podmínková proměnná“ samostatně:**
 - s monitorem nemůžeš omylem zapomenout zámek odemknout nebo přistoupit k datům bez ochrany;
 - data jsou *soukromá*, dostupná výhradně skrze metody monitoru → jasně vymezená kritická oblast a platnost invariantů.
 - V podstatě jde o **bezpečnější a samo-dokumentující obal** nad dvojicí „mutex + cond-var“.

4. Správa virtuální a fyzické paměti

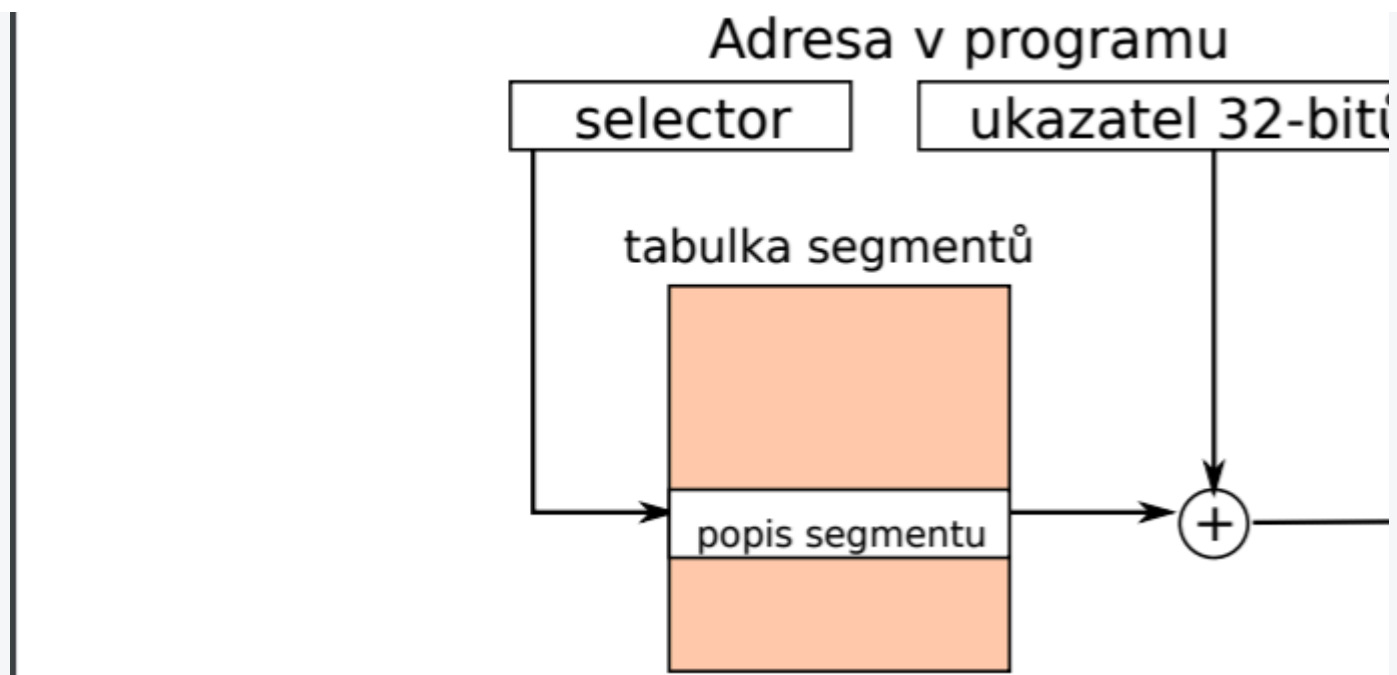
- co je a jak vypadá stránkovací tabulka, jaké jsou zásadní nevýhody stránkování, TLB (translation-lookaside-buffer), víceúrovňové stránkování v 32-bitovém a 64-bitovém systému, odkládání stránek na disk, algoritmy výběru oběti, metoda copy-on-write.

Názvosloví

- **FAP** – Fyzický adresní prostor, skutečná paměť počítače; její velikost je dána technickým limitem základní desky a kapacitou osazených RAM modulů.
- **LAP** – Logický (virtuální) adresní prostor, který vidí každý běžící proces.
 - Adresy v LAP jsou překládány OS na odpovídající místa ve FAP pomocí tabulek stránek a dalších struktur.
 - Velikost LAP závisí na architektuře CPU a OS:
 - 32-bit: 4 GiB (2^{32}) – většinou celý prostor nelze využít kvůli dělení jádra a uživatelského prostoru.
 - 64-bit: teoreticky 16 EiB (2^{64}), ale prakticky bývá omezeno (např. na 48 bitů = 256 TiB), hlavně kvůli velikosti TLB a implementaci stránkování.

Segmentace

- Segmentace je jedna z metod správy paměti, kde se paměť dělí na logické bloky – **segmenty** (např. pro kód, data, zásobník).
- Adresy v programu jsou tzv. **selektory**, které odkazují na konkrétní záznam v segmentové tabulce.
- Segmentový záznam obsahuje: základní adresu, délku segmentu, přístupová práva.
- Procesor pak fyzickou adresu vypočítá jako: `fyzická adresa = základna segmentu + offset`.



Výhody segmentace

- Délka segmentu odpovídá skutečné potřebě – **úspora paměti** (menší vnitřní fragmentace).
- Při přístupu mimo segment dojde k **výjimce** – typicky **segmentation fault**.
- Přesuny v paměti jsou **transparentní** – změna základny segmentu neovlivní kód procesu.
- Každý segment může mít samostatná **přístupová práva** – lepší ochrana paměti (např. nelze spustit data).

Nevýhody segmentace

- Alokace paměti je složitá – segmenty mají různou délku, takže jejich správné rozmístění není triviální.
- Při častém vytváření a rušení segmentů dochází k **externí fragmentaci** – malé mezery mezi segmenty nelze využít.
- Větší **režie při přístupu do paměti** – nutnost kontroly segmentového záznamu a výpočtu fyzické adresy.

Fragmentace

- *Externí (vnější)* – V paměti je dost místa, ale rozděleného na malé části – **nelze přidělit větší blok**.
- *Interní (vnitřní)* – Segment je větší, než proces reálně potřebuje → vzniká **nevyužitý zbytek** uvnitř segmentu.

Stránkování

Stránkování (paging) rozděluje logický adresní prostor procesu na pevně velké bloky – **stránky**, a fyzickou paměť na stejně velké **rámce** (frames).

- Překlad stránka → rámec zajišťují **stránkovací tabulky**, které spravuje jádro; hardwarová jednotka MMU je využívá k překladu adres.

Základní pojmy

- **Velikost stránky** – obvykle 4 KiB (x86/ARM), ale lze použít i větší: 2 MiB „huge-pages“, 1 GiB „gigapages“.
- **Page Frame Number (PFN)** – index rámce ve fyzické paměti.
- **Virtuální adresa** = ⟨index stránky, offset ve stránce⟩.
- **Page fault** – přerušení při přístupu na stránku, která není (zatím) namapována do RAM.

Stránkovací tabulka

- Každý proces má svou **stránkovací tabulku** – strukturu mapující virtuální adresy na fyzické rámce.
- Záznam o stránce obsahuje např.:
 - PFN cílového rámce (pokud je stránka přítomná),
 - přístupová práva (RWX),
 - příznaky (valid/dirty/accessed),

- info pro cache / TLB.
- V moderních architekturách (např. x86_64) je tabulka **víceúrovňová** (typicky 4 nebo 5 úrovní) – pro každou část adresy existuje jedna tabulka.

Eliminace externí fragmentace

- Fyzická paměť je rozdělena na **stejně velké rámce** – OS udržuje bitmapu nebo volný seznam.
- Každá stránka může být umístěna do libovolného rámce $\Rightarrow$ **nevznikají nespojitě díry** jako u segmentace.
- Vzniká pouze **vnitřní fragmentace** (max. 1 stránka na segment).
- Výběr volného rámce je efektivní (bitmapa – $O(1)$); není nutné relokovat paměť.

Překlad adresy – krok za krokem

1. **TLB lookup** – MMU nejprve hledá překlad (Virt $\rightarrow$ Fyz) v *Translation Lookaside Bufferu* (L1/L2 cache).
2. **Miss v TLB** $\Rightarrow$ hardware načte záznam ze stránkovací tabulky v paměti (víceúrovňový průchod podle částí adresy).
3. Pokud záznam říká, že stránka není v RAM $\Rightarrow$ **page fault**:
 - jádro zvolí volný rámec (nebo oběť, kterou swapne na disk),
 - načte požadovanou stránku (např. z binárky, souboru nebo swappu),
 - aktualizuje tabulku a TLB, zopakuje instrukci.

Nevýhody stránkování

- **Vícenásobný přístup** při TLB miss – může být třeba 4–5 čtení z paměti (1–4 úrovně tabulek + samotná data).
- **Vnitřní fragmentace** – poslední stránka segmentu často není plně využita.
- **TLB thrashing** – pokud pracovní set přesahuje kapacitu TLB, dochází ke zpomalujícím TLB missům.
- **Správa swappu** přináší I/O režii a může vést ke *thrashingu* celé RAM.

TLB (Translation Lookaside Buffer)

- Malá, **plně asociativní cache** překladů Virt $\rightarrow$ Fyz (desítky až stovky záznamů na CPU jádro).
- Výrazně zrychluje přístup – při TLB hit není třeba sahát na stránkovací tabulky.
- **TLB shutdown**: při změně mapování (např. `mmap()`, `fork()`) musí OS invalidovat TLB záznamy všech CPU, které daný proces používají – pomocí IPI.
- **ASID (Address Space ID)** nebo **PCID (Process Context ID)**:
 - značka v TLB, která umožňuje udržovat překlady i po přepnutí procesu,
 - zabráňuje nutnosti flush TLB při každém context switchi.

Víceúrovňové tabulky

Architektura	Úrovně překladu	Dekompozice virtuální adresy (bitové pole)
_____	_____	_____
x86 (32-bit)	2 (PDE, PTE)	10 bit index PDE • 10 bit index PTE • 12 bit offset
x86-64 (48bit LAP)	4 (PML4, PDPTE, PDE, PTE)	9 + 9 + 9 + 9 + 12
x86-64 (57bit LAP)	5 (PML5, PML4, PDPTE, PDE, PTE)	9 × 5 + 12

- Každá úroveň odpovídá stránkovací tabulce (page table), která se používá pro překlad části virtuální adresy.
- Jednotlivé části virtuální adresy slouží jako indexy do těchto tabulek – z každé se vybere záznam ukazující na tabulku další úrovně.
- Poslední úroveň obsahuje **Page Table Entry (PTE)** s fyzickým rámcem, ve kterém stránka začíná.
- Díky stránkování stránkovacích tabulek (samy jsou rozděleny na stránky) systém **alokuje jen části stromu**, které proces skutečně používá $\Rightarrow$ **úspora paměti**.
- Nejčastější hloubka tabulek dnes je 4 (x86-64), pátá úroveň se používá při větších LAP (např. 57bit v Linuxu).

Odkládání stránek na disk (swapping)

- **Swap area** – vyhrazená část disku (HDD/SSD), kam se ukládají stránky, které nejsou často používané, aby se uvolnila RAM.
- **Demand paging** – stránky se načítají do paměti až při pokusu o přístup → efektivnější využití RAM.
- **Thrashing** – situace, kdy proces spotřebuje tolik stránek, že dochází k častému swapování a výkon dramaticky klesá (systém více swapuje než počítá).

Algoritmy výběru oběti (page replacement)

- „Oběť“ = stránka ve fyzické paměti, kterou kernel vyhodí (nebo uloží do swapu), aby mohl přinést novou stránku (např. po page faultu).
- Cílem je vybrat stránku, která **nebude brzy znovu potřebná**, což je těžké odhadnout → používají se heuristiky.

Algoritmus	Idea	Poznámky
_____	_____	_____
FIFO	Vyhoď nejstarší stránku	jednoduchý; trpí Beladyho anomálií (větší paměť ⇒ horší výkon)
Second-Chance (Clock)	FIFO s dodatečným bitem „referenced“	základní Linuxová varianta (‘CLOCK-Pro’), stránka dostane „druhou šanci“
LRU / Approx. LRU	Vyhoď nejméně nedávno použitou	přesný LRU je drahý na implementaci; <u>OS</u> často používají aproximace (bitmapy, počítadla)
Working-Set	Drž stránky aktivní v posledním čase Δ	přesnější, ale dražší na výpočet
Adaptive (např. ARC)	Kombinace LRU a LFU	používá se v moderních <u>OS</u> (Windows, ZFS) i databázích

Copy-On-Write (COW)

- Mechanismus pro **efektivní sdílení paměti** mezi procesy.
- Při volání `fork()` nedochází hned ke kopírování všech stránek – místo toho:
 - Proces **rodíč i potomek sdílejí stejné rámce**, označené jako **read-only**.
 - Jakmile jeden z nich provede **zápis**, vyvolá se **page fault**.
 - Kernel:
 - alokuje nový rámec,
 - zkopíruje obsah původní stránky,
 - přemapuje novou stránku jako RW.
- Výhody:
 - výrazné **zrychlení `fork()`**, když následně proces provede `exec()`,
 - efektivní **sdílení statických dat** – např. kód (`.text`), knihovny apod.

Shrnutí výhod stránkování

- **Eliminace externí fragmentace** – všechny rámce mají stejnou velikost, takže OS vždy najde volný rámec bez „děr“; na rozdíl od segmentace není nutná kompakce.
- **Snadný růst haldy/stacku** – nové stránky se prostě namapují do dalších rámců.
- **Izolace procesů** – ochranné bity (R/W/X, User/Supervisor) zajišťují oddělení paměti mezi procesy.
- **Sdílení kódu a knihoven** – více procesů může mapovat stejný fyzický rámec (např. `.text` segment) do svého LAP.
- **Virtuální paměť > fyzická** – systém může využít swap a demand paging pro rozšíření dostupné paměti.
- **Podpora moderních technik** – Copy-On-Write, `mmap()` pro soubory, lazy allocation.

- Nevýhody* (víceúrovňový překlad, TLB missy, latence při swapu) se zmírňují pomocí:
 - větších TLB (ASID/PCID),
 - použití **huge-pages**,
 - lepších algoritmů výběru oběti,
 - dostatku fyzické paměti (RAM).

5. Souborové systémy

jaké typy souborových systémů znáte, který je vhodný pro sekvenční čtení a který pro náhodné čtení souborů. Vysvětlíte základní souborové systémy: FAT, systémy založené na inodech a systémy založené na extendech. Žurnálování – základní princip, kdy mohou vzniknout v souborovém systému chyby, jaké jsou úrovně žurnálování a jeho nevýhody.

Způsob organizace dat na disku – data jsou uložena v souborech, soubory jsou strukturované v adresářích (hierarchická struktura).

- Souborový systém určuje, jak jsou data fyzicky a logicky organizována, jak se přistupuje ke souborům, jaká metadata jsou vedena apod.

Příklady souborových systémů

- **FAT, FAT32** – jednoduché, široce kompatibilní (např. USB disky), bez žurnálování, náchylné na poškození při výpadku.
- **exFAT** – nástupce FAT32, podporuje větší soubory a disky, stále bez žurnálu.
- **NTFS** – moderní FS pro Windows, podporuje práva, šifrování, kompresi, žurnálování.
- **ext2/3/4** – nejpoužívanější na Linuxu; ext3/4 podporují žurnál, ext4 má rychlejší přístup a větší limity.
- **Btrfs** – pokročilý FS s podporou snapshotů, kontrolních součtů, RAIDu, dynamické alokace.
- **ZFS** – kombinace FS a správy disků, silné kontroly integrity dat, snapshoty, samoopravné mechanismy.

Možnosti uložení obsahu souboru

- **Souvislý úsek bloků**
 - Podobné jako alokace paměti – rychlý sekvenční přístup.
 - Nevýhoda: fragmentace a nutnost přesunů při zvětšení souboru.
 - Typicky vhodné pro **sekvenční čtení** (např. média, logy).
- **Spojové seznamy**
 - Každý blok obsahuje i ukazatel na další blok; adresář obsahuje odkaz na první blok.
 - Jednoduchá implementace, vhodné pro **sekvenční přístup**, ale:
 - Nevhodné pro náhodný přístup – nutné projít celý seznam.
 - Nemožnost přímého mapování souboru do paměti.
 - Jediný poškozený sektor může znepřístupnit celý soubor.
- **Indexové struktury**
 - Používá se samostatný **indexový blok**, který obsahuje ukazatele na datové bloky.
 - Umožňuje rychlý **náhodný přístup** (random access), stále dobrý i pro sekvenční čtení.
 - Při velkých souborech může být potřeba víceúrovňový index.
 - Používají např. ext a NTFS.

Základní souborové systémy

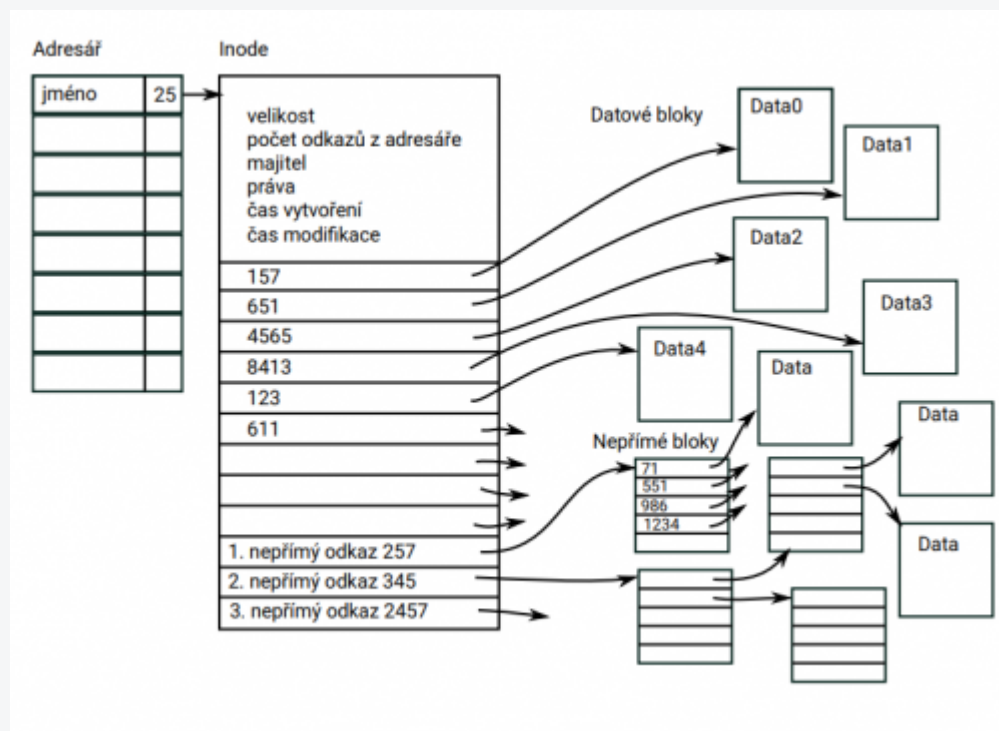
FAT (File Allocation Table)

- Starý, jednoduchý souborový systém s mnoha omezeními.
- Konstrukčně něco mezi spojovými seznamy a indexovou strukturou.
- Základní jednotka alokace je **cluster** (4–32 KiB).
- Maximální počet clusterů:
 - FAT16: 2^{16}
 - FAT32: 2^{28}

- exFAT: $2^{32} - 10$
- Disková struktura:
 - **MBR** (Master Boot Record) – informace o FS (velikost, jméno, počet FAT tabulek, apod.)
 - **FAT1, FAT2** – dvě redundantní tabulky
 - **Root directory**
 - **Data**
- V každé položce FAT je číslo následujícího clusteru nebo hodnota `-1` pro konec souboru.
- Nevýhody:
 - silná **fragmentace**
 - **sekvenční přístup** k FAT tabulce při čtení souboru
 - **omezená velikost souboru i disku**
 - chybí žurnálování

Inodový souborový systém

- Metadata o souborech jsou uložena ve **struktuře inode** (Index Node).
- Adresářová položka obsahuje jméno a číslo inode.
- Každý inode obsahuje:
 - několik přímých odkazů na datové bloky
 - nepřímé odkazy – jednoduchý, dvojitý, trojitý (pro větší soubory)
- Výpočet pozice bloku je snadný – dle offsetu v souboru.

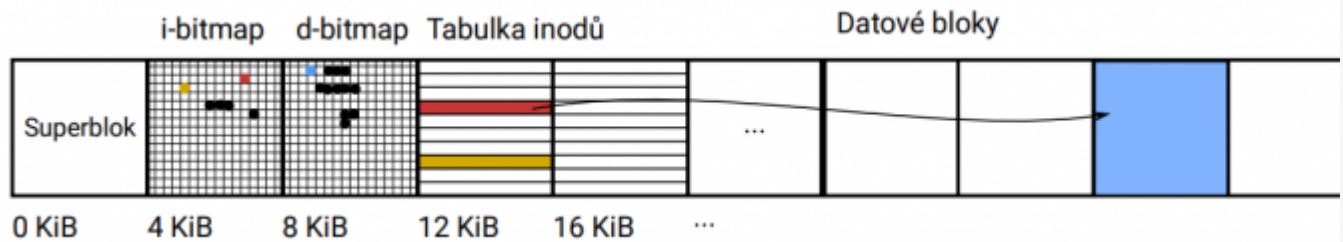


Rozložení na disku:

- **Pevný počet inode** – jsou uloženy v inode tabulce.
- **Superblok** – obsahuje informace o FS (velikost, počet inode, volné bloky, záložní superblok).
- **Kořenový adresář** – výchozí místo připojení FS.

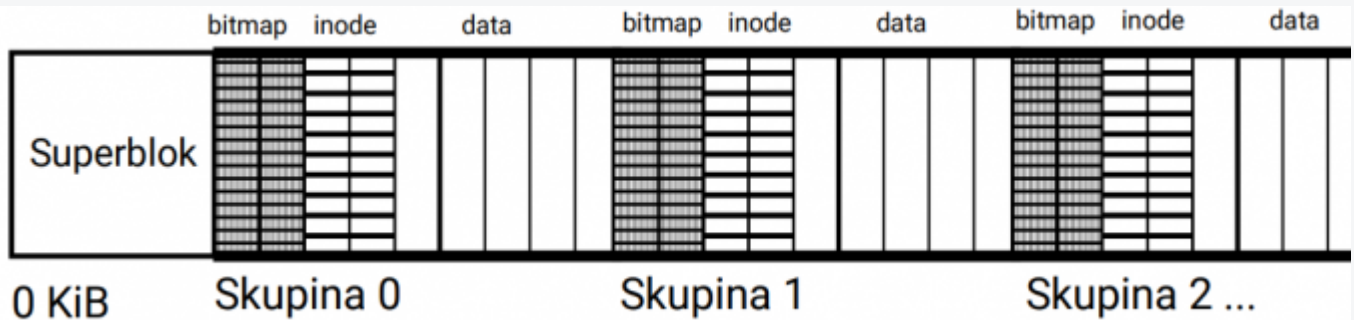
Hledání volného místa:

- Sekvenční procházení – neefektivní.
- Blok s nulou může být validní obsah souboru $\Rightarrow$ obtížné určit volný blok.
- Řešení:
 - **bitové mapy** (bitmaps) pro inode i datové bloky.
 - Vyhledávání je pak rychlé a efektivní.



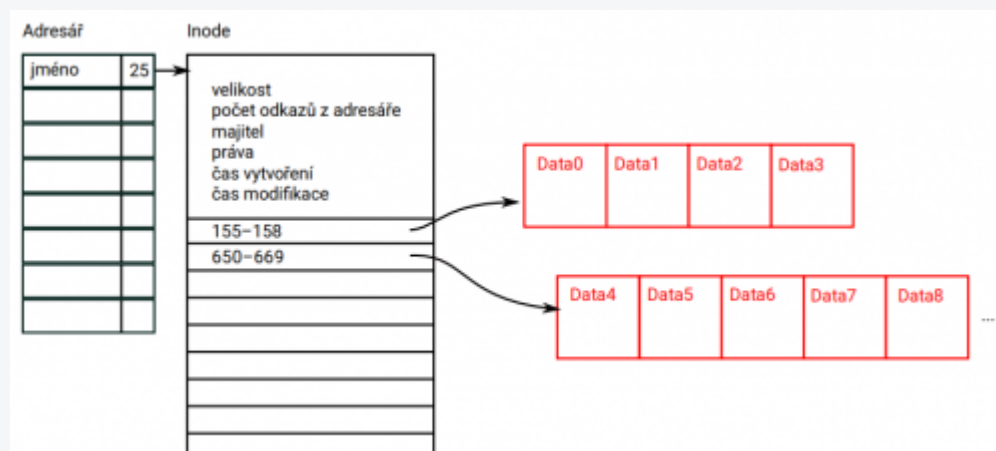
ext2/3/4:

- Při práci se souborem je potřeba přístup k bitmapě, inodu a datovým blokům.
- U rotačních disků je výhodné, když jsou bloky blízko sebe.
- FS dělí disk na **skupiny bloků**, které obsahují inody i data ⇒ snížení latence přístupu.



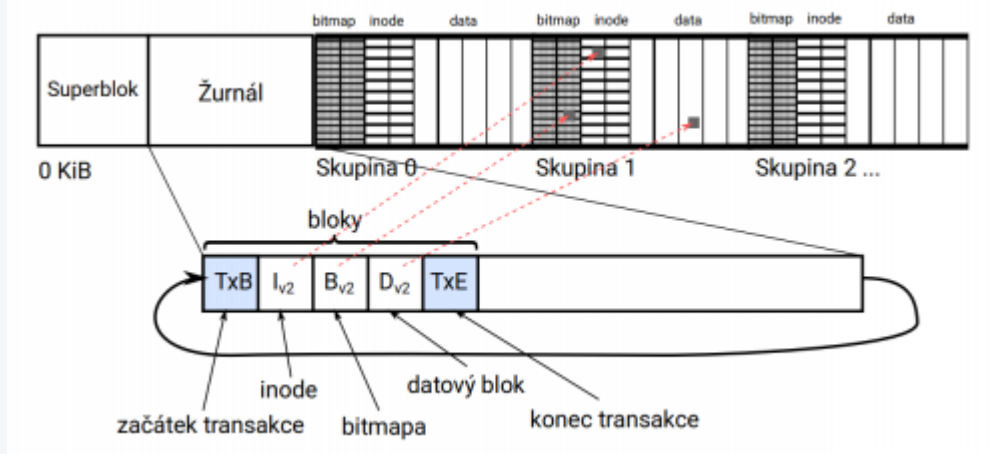
Extents

- U velkých souborů je indexace po jednotlivých blocích neefektivní.
- Moderní FS používají **extenty** – odkazy na **souvislé skupiny bloků** (namísto jednotlivých).
- Každý extent: ⟨začátek, délka⟩
- Výhody:
 - Méně režie než při uložení všech odkazů zvlášť.
 - Lepší výkon u sekvenčního i náhodného přístupu.
- Používá se např. v **ext4**, **NTFS**, **btrfs**.



Žurnálování

- Před tím, než se začne měnit obsah souborového systému, uloží se plánované změny do zvláštní oblasti disku – **žurnálu**.
- Pokud dojde k pádu systému, při startu **OS** se kontroluje žurnál a změny se podle něj **dokončí nebo zahodí** tak, aby byl FS v konzistentním stavu.
- Běžné v moderních FS: **NTFS**, **ext3**, **ext4**, **btrfs**, **ZFS** (ten má místo žurnálu transakční model se snapshoty).



Bezpečný postup změny FS (tzv. transakční protokol):

- **Commit** – ukončení transakce zápisem speciálního záznamu (`TxE`).
- **Checkpoint** – změny v inodech, datech, bitových mapách se zapíší na správná místa na disku.
- Po úspěšném dokončení se transakce **odstraní** ze žurnálu.

Scénáře při pádu systému

- **Pouze část transakce** zapsaná do žurnálu:
 - FS zůstává konzistentní, ale obsahuje původní (nezměněná) data.
 - OS při startu zjistí neúplnou transakci a ignoruje ji.
- **Celá transakce** je v žurnálu, ale **bloky na disku se nezmění**:
 - OS je použije k dokončení změn (tzv. *roll-forward*).
- **Transakce i bloky jsou zapsány**, ale transakce není odstraněna:
 - OS je znovu aplikuje (*idempotentní* operace – nezmění výsledek, když se provedou víckrát).
- **Pouze část transakce** – např. `TxB`, `I\_v2`, `TxE`, ale **chybí `B\_v2` a `D\_v2`**:
 - Riziko způsobeno optimalizacemi disku – **změna pořadí zápisů**.
 - OS musí použít tzv. **write barriers** (paměťové/IO bariéry), aby donutil disk dodržet pořadí:
 - Správná sekvence: `TxB`, `I\_v2`, `B\_v2`, `D\_v2`, *bariéra*, `TxE`.

* Nevýhody žurnálování:

- Režie navíc (dvojnásobné zápisy: nejprve do žurnálu, pak do FS).
- Složitost implementace.
- Zpomalení zápisových operací.

Flash paměti

- **Flash bloky** lze pouze přepsat, pokud jsou nejprve **vymazány**.
- **Zápis je jednorázový** – změna jednoho bajtu znamená smazání celého bloku (např. 4 MiB).
- **Omezený počet přepisů** – typicky 100 000 až 1 000 000 cyklů na blok.
- Často měněná data (bitmapy, FAT tabulky) mohou výrazně zkrátit životnost paměti.
- Důsledkem je **write amplification** – malá změna vyžaduje velkou operaci.

Řešení pro flash paměti

- Nepoužívají se přímo, ale s řadičem, který implementuje **Flash Translation Layer (FTL)**:
 - Skryje omezení zápisu, přemapovává fyzické bloky, spravuje wear-leveling.
 - Typické u SSD, eMMC, SD karet.
- **Speciální souborové systémy pro flash**:
 - **UBIFS** – pro NAND flash, umí journal, garbage collection.
 - **JFFS2** – log-strukturovaný, pro malé embedded systémy.
 - **NILFS** – log-based, kontinuální snapshoty, optimalizovaný pro SSD.

6. Bezpečnost

- co je Trusted Computing Base, základní metody řízení přístupu, jak se provádí útok na přetečení zásobníku, jak se lze takovému útoku bránit.

Trusted Computing Base (TCB)

- Množina všech komponent, kterým systém **musí důvěřovat**, aby zaručil bezpečnost.
- Pokud selže některá část TCB, **celý systém může být kompromitován**.
- Příklady: hardware, jádro OS, správce systému (root), hypervisor, firmware, kryptografické klíče.

Základní metody řízení přístupu

- Základním teoretickým modelem je **matice řízení přístupu** – popisuje, kdo má jaká oprávnění k čemu v daném čase.

	Obj1	Obj2	Obj3	Subj2
Subj1 (alice)	R	RW		send
Subj2 (bob)		RX		control
Subj3 (root)	RW		RWX	recv

- **Subjekty** – např. uživatelé, procesy.
- **Objekty** – např. soubory, zařízení, síťové sockety.
- Subjekty mohou být zároveň i objekty (např. pokud lze nastavit práva k procesu).

Ukládání stavu ochrany není typicky jako matice (moc „řádké“, neefektivní, dynamické). V praxi 2 zřejmé volby:

- Ukládání jednotlivých sloupců dohromady s objektem
 1. Každý sloupec je nazýván „seznam pro řízení přístupu“ (**access control list**, ACL) daného

objektu

- Ukládání jednotlivých řádků dohromady se subjektem
 1. Definuje objekty, ke kterým má daný subjekt přístup – doména ochrany (**protection domain**)

daného subjektu

1. Každý takový řádek je nazýván „seznam schopností“ (**capability list**)

Seznamy pro řízení přístupu (ACL)

- Nejrozšířenější model, implementován v UNIX, Linux, Windows...
- Subjekty bývají seskupeny do tříd (např. majitel, skupina, ostatní).

```
$ ls -ld /var/spool/cups
drwx--x--- 1 root lp 6754 Nov 22 00:00 /var/spool/cups
```

- Moderní systémy (Linux, Windows) podporují **rozšířené ACL** – více individuálních pravidel.
- Podpora **negativních oprávnění** – např. zakázat přístup členům určité skupiny.
- **Metaoprávnění** – např. oprávnění měnit ACL nebo členství ve skupinách.

Obj1	
Subj1	R
Subj2	
Subj3	RW

Seznamy schopností (Capability Lists)

- **Schopnost (capability)** = token, který **pojmenovává objekt a uděluje oprávnění**.
- Držitel schopnosti může s objektem manipulovat podle udělených práv.
- Nevyžaduje centrální seznam přístupů – **decentralizovaný model přístupu**.

Subj1 (alice)	Obj1	Obj2	Obj3	Subj2
	R	RW		send

- Typicky se používá v systémech, kde se upřednostňuje bezpečnostní izolace a modularita (např. mikrojádra, sandboxy, některé OS pro embedded zařízení).
- Objekty jsou **neviditelné** bez příslušné schopnosti ⇒ důležitý princip **least privilege**.

ACL vs Capability List

Seznamy řízení přístupu (<u>ACL</u>)	Schopnosti (Capabilities)
_____	_____
Tradiční model; proces musí vědět, jaký objekt chce , a systém mu řekne, jestli má přístup.	Objekty jsou přístupné pouze skrze předané schopnosti – uživatel je <i>*nemůže najít sám*</i> .
Oprávnění určena podle identity subjektu (UID, skupina, ...).	Proces dostane pouze schopnosti , které mu byly explicitně předány (např. přes socket, fork, init).
Problém ambientní autority – proces má automaticky všechna práva uživatele.	Neexistuje ambientní autorita – práva se předávají cíleně a omezeně.
Nelze snadno omezit práva potomků (děděná práva).	Nikdo nemůže předat právo, které sám nemá .
Linux částečně řeší pomocí namespaces , ale není to univerzální ani bezpečnostně dokonalé.	Princip <i>*objektově orientované bezpečnosti*</i> , čistší a škálovatelnější řešení.

Stack Overflow – how to attack

- **Přetečení zásobníku** nastane, když program zapíše více dat do bufferu (pole, proměnné) na zásobníku, než je jeho velikost.
 - ⇒ přepíší se další položky na zásobníku – např. sousední proměnné, uložený rámec (frame pointer) nebo **návratová adresa funkce**.
- Útočník může do bufferu vložit tzv. **shellcode** – malý strojový kód, který může např. spustit shell a umožnit vzdálenou kontrolu systému.
- Pokud útočník přepíše **návratovou adresu**, může přesměrovat běh programu na vlastní kód (shellcode) nebo jinou část programu.
- Útoky se zaměřují na programy v C/C++, kde chybí **automatická kontrola hranic polí**.
- **Omezení shellcodu** – nesmí obsahovat binární nuly (`\0`), protože řetězce jsou jimi ukončovány. ⇒ používají se techniky kódování.
- Moderní techniky: **Return-Oriented Programming (ROP)** – nevpučují nový kód, ale skládají útočný řetězec z již existujících instrukcí v programu (**gadgety**).

Stack Overflow – how to defend

- **Nespustitelný zásobník (NX, XD bit, ARM UXN)** – zakáže spuštění kódu z datové části zásobníku.
- **ASLR (Address Space Layout Randomization)** – náhodné rozmístění paměťových oblastí (zásobník, knihovny, binárky) při každém spuštění programu.
- **Stack Canary / Stack Protector** – před návratovou adresu se vloží kontrolní hodnota (canary). Pokud je přepsaná, program detekuje útok a ukončí se.

- **Retguard** – návratová adresa je zakódována při vstupu do funkce a dekodována při návratu. Změna adresy útočníkem způsobí pád.
- **Bezpečné programování** – validace velikosti vstupních dat, používání bezpečných funkcí (``strncpy`` místo ``strcpy``, ``snprintf`` místo ``sprintf``).
- **Používání moderních jazyků a knihoven**, které mají ochranu proti přetečení zabudovanou (Rust, některé safe C knihovny).
- **Oddělení práv a minimalizace oprávnění** – běžící procesy by měly mít minimální potřebná oprávnění, aby případný útok měl omezený dopad.

7. Virtualizace

- softwarová virtualizace, metoda trap-and-emulate, virtualizace systémového volání, virtualizace stránkových tabulek, hardwarově asistovaná virtualizace.

Virtualizace je **abstrakce hardwaru** a jeho částečná emulace v softwaru. Hostujícímu systému je vytvořena iluze, že běží na vlastním fyzickém stroji.

Základní pojmy:

- **Hostitel (host)** – fyzický hardware nebo virtualizované prostředí, na kterém běží hypervizor.
- **Hypervizor** – software, který emuluje virtuální hardware a řídí běh VM.
- **Host (guest)** – OS běžící na virtuálním hardware poskytovaném hypervizorem.

Typy virtualizace:

- **Plná virtualizace** – host neví, že běží na virtualizovaném systému (typicky trap-and-emulate).
- **Paravirtualizace** – host si je vědom virtualizace a aktivně spolupracuje s hypervizorem.
- **Emulace** – celé prostředí je softwarově emulované, včetně instrukcí CPU (např. QEMU bez KVM).
- **Cloud virtualizace** – vzdálené řízení a spouštění VM v rámci cloud infrastruktury.

Výhody VM

- **Izolace** – VM jsou navzájem i od hosta oddělené, vyšší bezpečnost.
- **Současný běh více OS** – např. Linux + Windows současně na jednom stroji.
- **Přenositelnost a snapshoty** – běžící VM lze přesunout nebo pozastavit.
- **Vývoj a testování** – chyby v kernelu nezhodí celý systém.

Problémy virtualizace

Moderní CPU má dva režimy:

- **Uživatelský režim (user mode)** – omezený, aplikace, nelze přistupovat k HW.
- **Privilegovaný režim (kernel mode)** – OS, přístup ke všemu v systému.

VM nesmí běžet přímo v privilegovaném režimu – vnímáno by to jako bezpečnostní riziko. VM tedy běží v uživatelském režimu hostitele ⇒ je nutné emulovat vlastní **uživatelský a privilegovaný režim** uvnitř VM.

Trap and Emulate

- Instrukce v **uživatelském režimu** hosta se vykonávají nativně.
- Pokus o vykonání **privilegované instrukce** (např. přístup k HW) vyvolá **výjimku** (trap).
- Výjimku zachytí hypervizor, provede požadovanou akci a vrátí řízení zpět do VM.
- Hostující OS si myslí, že vše proběhlo nativně – **transparentní virtualizace**.
- Nutné rozlišovat:
 - **Privilegované instrukce** – způsobí trap, když jsou volány z uživatelského režimu.
 - **Citlivé instrukce** – mění globální stav nebo závisí na systému ⇒ musí být zachytitelné (v ideálním případě vždy privilegované).

Virtualizace systémových volání

- Systémová volání (např. ``open``, ``read``, ``write``) přecházejí z uživatelského do privilegovaného režimu.
- Ve VM se volání spustí pomocí **trap**, který hypervizor zachytí a provede požadovanou činnost.
- ⇒ **lehce pomalejší** než nativní syscalls, ale zachována kompatibilita a izolace.

Virtualizace stránkových tabulek

- Každý OS spravuje své **virtuální paměťové mapování** pomocí vlastních stránkových tabulek.
- Hypervizor ale musí zajistit, že host nemůže měnit skutečné fyzické mapování:
 - Host dostane **virtuální stránkové tabulky pouze pro čtení**.
 - Pokus o změnu vyvolá **trap** – hypervizor provede potřebnou aktualizaci.
- Efektivnější varianta: **shadow page tables** – hypervizor vede vlastní mapu překladů, kterou host nevidí.
- Modernější způsob: **EPT (Intel)** / **NPT (AMD)** – viz níže.

Hardwarově asistovaná virtualizace

Moderní procesory (Intel VT-x, AMD-V) přidávají nový režim:

- **Non-root execution mode** – speciální režim pro běh VM.
- Privilegované instrukce v tomto režimu mohou běžet přímo bez nutnosti trap-emulace.
- Významně zvyšuje výkon, protože:
 - většina běžných činností běží nativně,
 - trapy nastávají jen při I/O nebo specifických výjimkách.

Navíc podporují:

- **EPT (Extended Page Tables)** – hardwarová správa překladů virtuální → fyzická (host) → skutečná fyzická (hostitel) adresa.
- **Nested virtualization** – virtualizace uvnitř virtualizace (VM v jiné VM).

Shrnutí

Oblast	Princip	
_____	_____	
Trap and Emulate	Hypervizor zachytává výjimky při pokusu o privilegovanou instrukci.	
Virtualizace syscalls	Přes trap do hypervizoru, který syscall provede za VM.	
Virtualizace page tables	Host spravuje své PT, ale nesmí je měnit ⇒ trapy a ochrana zápisu.	
HW asistence	Non-root režim + EPT/NPT – mnoho operací běží přímo na CPU.	

Trace: • [b4b35osy](#)